DSA ASSIGNMENT

Exercise 7: Financial Forecasting

1.Understand Recursive Algorithms:

a. Explain the concept of recursion and how it can simplify certain problems.

Recursion is a programming technique in which a function calls itself repeatedly to solve a problem by breaking it into smaller, similar sub-problems. The process continues until a **base case** is reached, which can be solved directly without further recursion.

In financial forecasting, recursion is useful because the future value for a given period depends on the value from the previous period. This creates a repeating pattern that can be solved recursively.

- Recursive relation: **FV(n) = FV(n − 1) × (1 + growthRate)**
- Base case: **FV(0) = Present Value**

Thus, recursion provides a simple and efficient way to calculate future values over multiple periods by repeatedly applying the growth rate to the previous period's value.

2. Setup & Implementation:

- **Create a method to calculate the future value using a recursive approach.**
- **Implement a recursive algorithm to predict future values based on past growth rates.**

CODE:

FinancialForecasting.java

```
package MOD_2.exercise_7;


import java.util.Scanner;


public class FinancialForecasting {
   public static double futureValue(double presentValue, double growthRate, int periods){
      if(periods==0){
         return presentValue;
```

```
    }
    return futureValue(presentValue*(1+growthRate), growthRate, periods-1);
  }
  public static void main(String args[]){
    Scanner sc=new Scanner(System.in);
    System.out.println("Enter principle: ");
    double principal = sc.nextDouble();
    System.out.println("Enter rate: ");
    double rate = sc.nextDouble();
    System.out.println("Enter years: ");
    int years = sc.nextInt();

    double result = futureValue(principal, rate, years);
    System.out.printf("Future Value after %d years: $%.2f%n", years, result);

    System.out.println("\nYear-by-Year Breakdown:");
    for (int y = 0; y <= years; y++) {
      System.out.printf("Year %d: $%.2f%n", y, futureValue(principal, rate, y));
    }
  }
}
```

OUTPUT:

```
Enter principle:
10000
Enter rate:
0.08
Enter years:
5
Future Value after 5 years: $14693.28

Year-by-Year Breakdown:
Year 0: $10000.00
Year 1: $10800.00
Year 2: $11664.00
Year 3: $12597.12
Year 4: $13604.89
Year 5: $14693.28
```

4. Analysis:

a. Discuss the time complexity of your recursive algorithm.

The recursive algorithm makes one recursive call for each period (year) until the base case (periods == 0) is reached. If there are n periods, the function is called n + 1 times (including the base case).

- **Time Complexity: O(n)**
- **Space Complexity: O(n)**

The time complexity is O(n) because the algorithm performs a constant amount of work in each recursive call and makes n recursive calls. The space complexity is also O(n) due to the recursion call stack, which stores one stack frame for each recursive call until the base case is reached.

b. Explain how to optimize the recursive solution to avoid excessive computation.

The current recursive solution has a time complexity of O(n) and does not perform redundant calculations when computing a single future value. However, if the same values are calculated repeatedly, excessive computation can occur.

One way to optimize recursion is by using memoization, where previously computed results are stored in a cache (such as an array or HashMap). Before performing a recursive call, the algorithm checks whether the result has already been computed. If it exists, the stored value is returned directly, avoiding unnecessary calculations.

Another optimization is to replace recursion with an iterative approach, which eliminates the recursive call stack and reduces the space complexity from O(n) to O(1).

For financial forecasting, an even more efficient approach is to use the compound interest formula:

$$FV = PV \times (1 + r)^n$$

where:

- **FV** = Future Value
- **PV** = Present Value
- **r** = Growth Rate
- **n** = Number of Periods